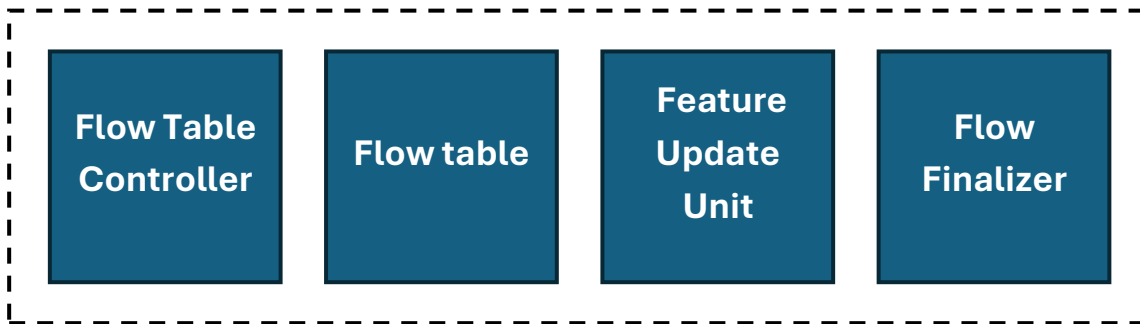


Blocos do sistema de Extração de Features de Fluxo



1. Entradas do sistema

O sistema receberá os metadados produzidos pelas etapas anteriores:

- **flow_key:** chave utilizada para identificar um fluxo, formada pela 5-tupla {ip_origem, ip_destino, porta_origem, porta_destino, protocolo}. Inicialmente serão considerados **uniflows**, de modo que uma comunicação $A \rightarrow B$ será tratada como diferente de $B \rightarrow A$.
- **hash_index:** índice calculado pelo módulo de hash a partir da flow_key. Será utilizado como endereço para consulta da Flow Table.
- **packet_length:** tamanho do pacote recebido. Será utilizado na atualização do número total de bytes observados no fluxo (byte_count).
- **timestamp:** instante de chegada do pacote. Será utilizado para registrar os timestamps do primeiro e do último pacote do fluxo para posterior cálculo de features temporais.
- **valid:** sinal de controle que indica que os metadados apresentados na entrada correspondem a um novo pacote válido e podem ser processados.

A aceitação de um novo conjunto de metadados será controlada pelo sinal ready, gerado pelo Flow Table Controller.

2. Modelamento da Flow Table

A **Flow Table** será responsável pelo armazenamento do estado dos fluxos monitorados. Cada posição da tabela representará um fluxo e armazenará seu registro completo:

- **valid:** indica se a posição da tabela está atualmente ocupada.
- **flow_key:** chave utilizada para identificar o fluxo.
- **packet_count:** quantidade de pacotes observados no fluxo.
- **byte_count:** quantidade total de bytes observados no fluxo.
- **first_timestamp:** instante de chegada do primeiro pacote do fluxo.

- last_timestamp: instante de chegada do pacote mais recente do fluxo.

Uma operação de leitura, realizada a partir do hash_index, retornará o **registro completo da posição consultada**. Dessa forma, uma única leitura fornecerá ao Flow Table Controller tanto as informações necessárias à classificação da consulta (valid e flow_key) quanto o estado acumulado das features.

3. Flow Table Controller: HIT, MISS e COLLISION

Considere como exemplo a chegada de um pacote com:

- flow_key = A;
- hash_index = 37.

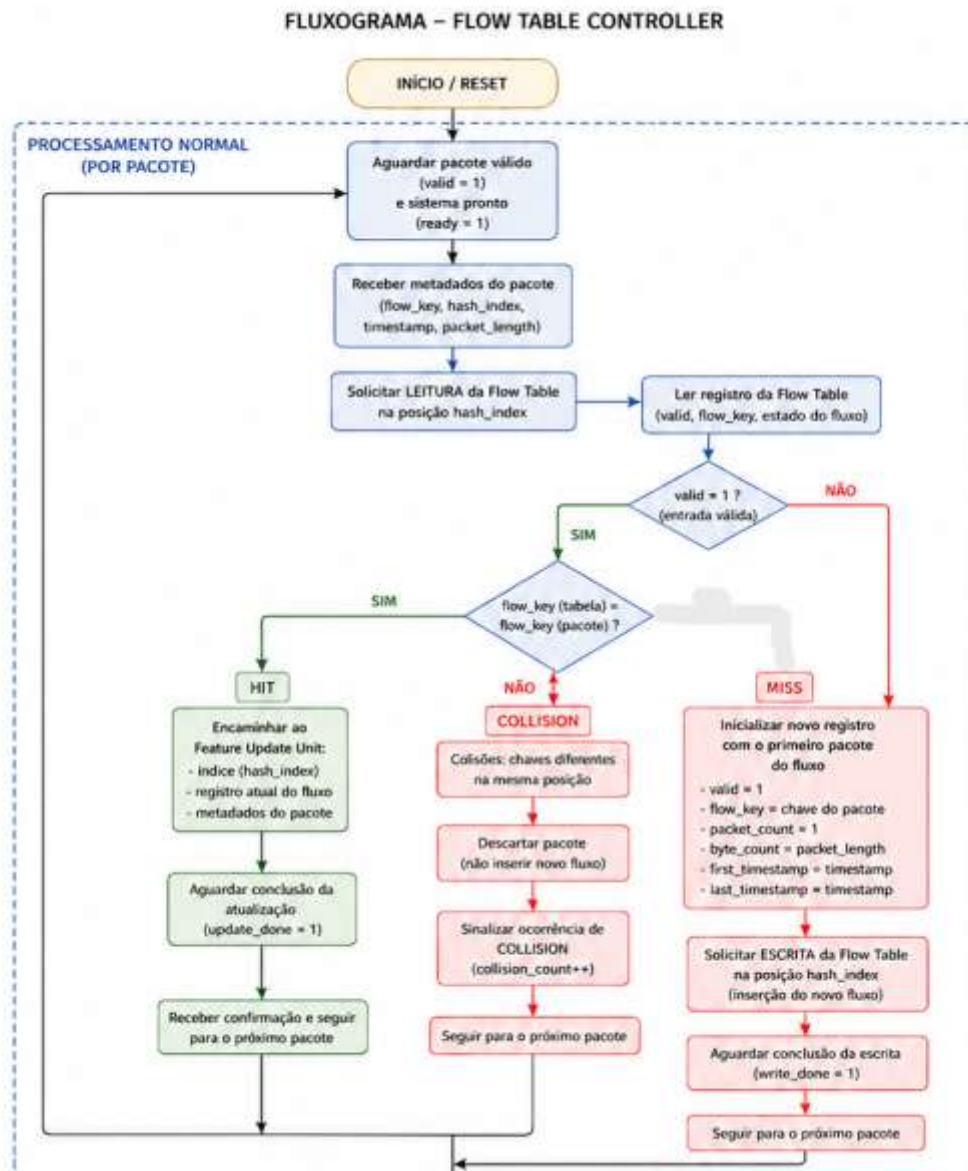
O processamento básico será:

- a) O **Flow Table Controller** solicita um LOOKUP da posição 37 da Flow Table.
- b) A Flow Table retorna o registro completo armazenado nessa posição.
- c) O Controller verifica o campo valid e compara a flow_key armazenada com a flow_key recebida.

A partir dessa análise, três situações podem ocorrer:

- a) **MISS**, caso valid = 0:
 - A posição encontra-se disponível e um novo fluxo deverá ser inserido.
 - O Flow Table Controller inicializará a entrada com:
 - valid = 1;
 - flow_key = incoming_flow_key;
 - packet_count = 1;
 - byte_count = packet_length;
 - first_timestamp = timestamp;
 - last_timestamp = timestamp.
 - Em seguida, solicitará a escrita da nova entrada na Flow Table.
- b) **HIT**, caso valid = 1 e stored_flow_key = incoming_flow_key:
 - O fluxo já se encontra armazenado na tabela.
 - O Flow Table Controller encaminhará ao **Feature Update Unit**:
 - o registro atual completo do fluxo;
 - o hash_index;
 - o packet_length;
 - o timestamp;
 - uma requisição de atualização.

- O Feature Update Unit calculará o novo estado e solicitará sua escrita na Flow Table. Ao término da operação, sinalizará done ao Flow Table Controller.
- c) **COLLISION**, caso: `valid = 1` e `stored_flow_key ≠ incoming_flow_key`:
- Duas chaves diferentes terão sido associadas ao mesmo índice, caracterizando uma colisão.
 - Na primeira prova de conceito, a colisão será sinalizada e o novo fluxo não será inserido na tabela.



4. Tratamento de colisões

Considere:

- Flow A → `hash_index = 25`;
- Flow B → `hash_index = 25`.

Caso a posição 25 já esteja ocupada pelo Flow A quando um pacote do Flow B chegar, ocorrerá uma colisão.

Algumas estratégias possíveis tratamento de colisões são:

- a) **Drop:** O novo fluxo não é inserido na tabela. A ocorrência pode ser sinalizada e contabilizada para fins de verificação. É a estratégia mais simples e será utilizada inicialmente na prova de conceito.
- b) **Linear probing:** Quando a posição indicada pelo hash estiver ocupada por outro fluxo, o controlador poderá consultar sequencialmente as posições seguintes até encontrar a chave procurada ou encontrar uma posição livre. A estratégia apresenta implementação relativamente simples, porém o número de acessos à memória e a latência tornam-se dependentes da quantidade de colisões.
- c) **Buckets / Set associative:** Cada resultado do hash poderá selecionar um pequeno conjunto de entradas candidatas. Exemplo: hash 25 → entry 25A, entry 25B, entry 25C, entry 25D. O fluxo será procurado entre as entradas associadas ao índice. Essa estratégia limita o número de posições verificadas, mas exige maior quantidade de memória por índice e lógica adicional de comparação.
- d) **Cuckoo Hashing:** São utilizadas duas ou mais funções hash, disponibilizando múltiplas posições candidatas para uma mesma chave. Caso uma das posições esteja disponível, o fluxo pode ser inserido diretamente. Caso todas estejam ocupadas, uma entrada existente pode ser realocada para uma de suas posições alternativas. Embora permita melhor utilização da tabela e possa oferecer alto desempenho, essa técnica apresenta maior complexidade de controle.

5. Pacotes consecutivos e backpressure

Considere dois pacotes consecutivos pertencentes ao mesmo fluxo.

O processamento de um HIT envolve uma operação de: READ → UPDATE → WRITE

O Flow Table Controller realiza a leitura do estado atual, o Feature Update Unit calcula o novo estado e o resultado é escrito novamente na Flow Table.

Caso um segundo pacote do mesmo fluxo seja aceito antes da conclusão da primeira atualização, poderá ocorrer um **data hazard**.

Exemplo:

Considerando packet_count = 10:

- Pacote 1 lê 10 e pacote 2 também lê 10;
- Pacote 1 escreve 11 e pacote 2 escreve 11;
- O valor final será 11, quando deveria ser 12.

Na primeira implementação, esse problema será evitado por meio de **backpressure**.

Assim, enquanto um pacote estiver sendo processado:

- ready = 0;
- Nenhum novo pacote será aceito.

Após a conclusão da operação:

- ready = 1;
- O sistema poderá aceitar o próximo pacote.

Essa solução reduz o throughput máximo da arquitetura, porém simplifica significativamente a lógica de controle e garante a consistência dos estados armazenados.

Em uma evolução posterior, poderão ser estudadas técnicas de pipeline, detecção de dependências e encaminhamento de dados para permitir maior paralelismo.

6. Dimensionamento da memória da Flow Table

Deverá ser definido o número máximo de fluxos que poderão permanecer simultaneamente armazenados na tabela. É conveniente utilizar uma quantidade de entradas que seja potência de dois, facilitando a utilização direta do resultado do hash como índice.

Por exemplo:

- N = 1024 entradas;
- $\log_2(1024) = 10$ bits;
- O hash_index possuirá 10 bits.

O consumo aproximado de memória será dado por:

Memória total = número de entradas × largura de cada entrada

O dimensionamento definitivo deverá considerar:

- capacidade da FPGA utilizada;
- quantidade de BRAM disponível;
- largura de cada registro;
- quantidade esperada de fluxos simultaneamente ativos.

Durante a fase inicial de simulação, será utilizada uma tabela reduzida, por exemplo com 16 ou 32 entradas, facilitando a inspeção e validação do seu funcionamento.

Posteriormente, a tabela poderá ser dimensionada para centenas ou milhares de entradas e adaptada para utilização eficiente das memórias internas da FPGA.

7. Representação do timestamp

O timestamp representará o instante de chegada de cada pacote. Na primeira fase, em que os pacotes serão obtidos de arquivos PCAP, esse valor será fornecido diretamente pelo ambiente de verificação. Para cada fluxo serão armazenados:

- first_timestamp: timestamp correspondente ao primeiro pacote;
- last_timestamp: timestamp correspondente ao pacote mais recente.

Esses valores permitirão posteriormente calcular: $\text{duration} = \text{last_timestamp} - \text{first_timestamp}$

A unidade e a largura utilizadas para representar o timestamp deverão ser compatíveis com a resolução temporal do ambiente de verificação e com os recursos disponíveis na FPGA.

8. Dimensionamento das entradas da Flow Table

A largura de cada registro da tabela dependerá dos campos armazenados. Considerando IPv4:

- valid: 1 bit
- src_ip: 32 bits
- dst_ip: 32 bits
- src_port: 16 bits
- dst_port: 16 bits
- protocol: 8 bits
- packet_count: a definir
- byte_count: a definir
- first_timestamp: a definir
- last_timestamp: a definir

A flow_key, isoladamente, possui: $32 + 32 + 16 + 16 + 8 = 104$ bits

A largura dos contadores e dos timestamps deverá ser escolhida considerando os maiores valores esperados durante o processamento.

A largura total de cada registro será então utilizada, juntamente com o número de entradas da Flow Table, para estimar o consumo de memória da implementação.

9. Feature Update Unit

O **Feature Update Unit** será responsável por calcular o novo estado de um fluxo quando o Flow Table Controller identificar um HIT.

Para cada novo pacote serão realizadas, inicialmente, as seguintes atualizações:

- `packet_count_new = packet_count + 1;`
- `byte_count_new = byte_count + packet_length;`
- `first_timestamp_new = first_timestamp;`
- `last_timestamp_new = timestamp.`

O estado atualizado será então escrito novamente na Flow Table.

Após a conclusão da escrita, o Feature Update Unit sinaliza done ao Flow Table Controller.

10. Finalização do fluxo

Será necessário definir quando o estado acumulado de um fluxo deverá ser convertido no vetor final de features.

Entre os possíveis critérios de finalização estão:

- Encerramento explícito de uma conexão TCP:** Pacotes contendo flags como FIN ou RST podem indicar o encerramento de uma conexão TCP. Essa estratégia, entretanto, não atende diretamente fluxos UDP e pode não capturar conexões TCP encerradas de forma anormal.
- Timeout de inatividade:** Um fluxo pode ser considerado encerrado quando permanecer sem receber novos pacotes durante determinado intervalo. Essa estratégia pode ser utilizada tanto para TCP quanto para UDP, porém exige que as entradas da Flow Table sejam verificadas periodicamente durante o processamento, aumentando a complexidade da lógica de controle e do acesso à memória.
- Finalização ao término do PCAP:** Na primeira prova de conceito, será utilizada a finalização ao término do arquivo PCAP.

Quando o ambiente de verificação sinalizar o fim dos pacotes, o Flow Table Controller deixará de aceitar novas entradas (`ready = 0`) e percorrerá sequencialmente as posições da Flow Table.

Para cada posição:

- O Controller realizará um LOOKUP;
- Receberá o registro completo;
- Verificará o campo `valid`;
- Caso a entrada esteja ocupada, encaminhará o registro ao **Flow Finalizer**.

O Flow Finalizer utilizará o estado acumulado para calcular as features finais e organizará os valores no vetor de features disponibilizado na saída do sistema.

Após a geração do vetor, o Flow Finalizer sinalizará done ao Flow Table Controller. O Controller poderá então invalidar a entrada correspondente e avançar para a próxima posição da tabela.

O processo será repetido até que todas as entradas da Flow Table tenham sido verificadas.

Essa estratégia reduz significativamente a complexidade da primeira implementação e permite validar o funcionamento completo da arquitetura antes da inclusão de mecanismos de finalização por timeout ou eventos de protocolo.